

Block Intelligence Report

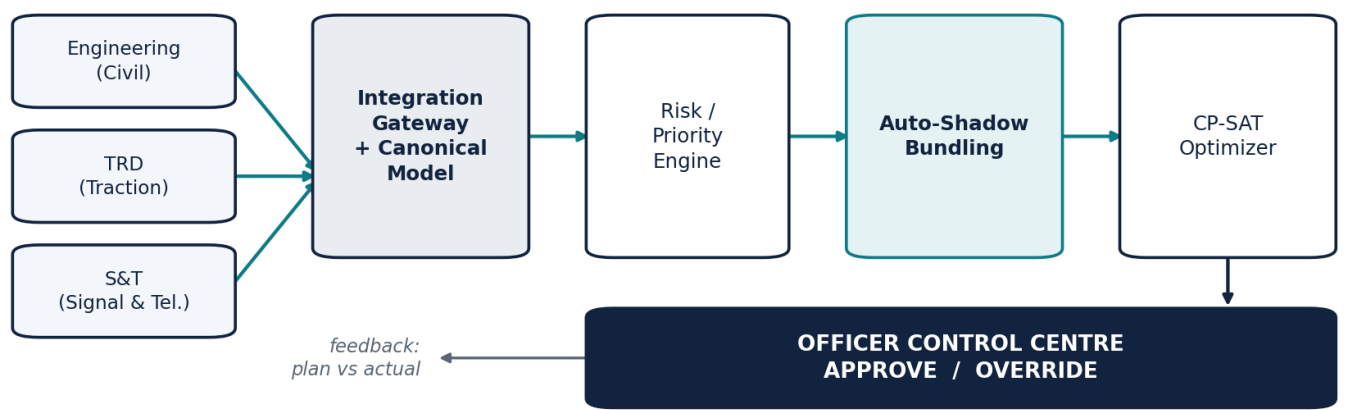
Automated cross-department railway maintenance block planning for Engineering, TRD and S&T | Team Ithaca | architecture, simulations and build plan

Executive decision

Build an **intelligence layer**, not another railway system. TMS, SMMS, TDMS, COA and BDMS remain the operational sources and the officer's workflow. On top of them we add one shared canonical railway model, a risk engine, proactive cross-department bundling (Auto-Shadow), CP-SAT scheduling, officer approval and a plan-vs-actual feedback loop.

The real problem is not fetching data. It is reconciling three departments' records onto the **same physical corridor, KM range and time window**, and then using that shared view to stop the same stretch of track being taken three separate times.

How the system works end to end



Three departmental sources are normalised into one canonical model, scored for risk, bundled where compatible, scheduled by a deterministic optimiser, and approved by the officer.

The seven decisions that define the project

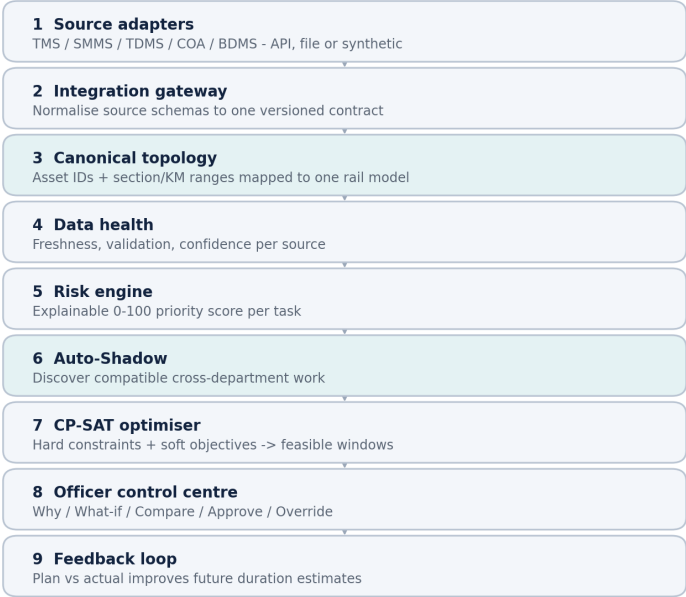
Decision area	Final choice	Why
System position	Intelligence layer above existing systems	Lowest adoption friction; departments keep their workflow
Data acquisition	Adapter gateway: API + export + synthetic	No imaginary production endpoints; one contract in all modes
Cross-system identity	Canonical topology + asset/KM mapping	Solves the real integration problem, enables bundling
Role of AI	Risk, urgency, failure and duration prediction	Prediction where it adds value, without safety authority
Scheduling	Google OR-Tools CP-SAT	Deterministic hard constraints, explainable optimisation
Human role	Officer approves or overrides	Operational authority stays with the controller
Headline USP	Auto-Shadow Intelligence	One possession covers compatible work from three departments

AI predicts. Optimisation guarantees feasibility. The officer approves.

1. Problem to product: what we are actually solving

Engineering / Civil, Traction Distribution (TRD) and Signal & Telecom (S&T) each request possessions independently. Without a shared cross-department view two things go wrong: compatible jobs on the same stretch create repeated outages, and lower-priority work keeps sliding until its risk quietly grows. The deliverable is therefore a **coordination and prioritisation layer**, not a replacement scheduler.

The nine build stages



The critical distinction

Fetching is not integration. TMS, SMMS, TDMS and COA can each hand over useful data while still using different schemas, identifiers and refresh rates. That is why an adapter layer, a canonical rail model and an overlap-resolution step sit between the sources and the planner.

Adoption principle

Compatibility over replacement. Officers should experience the prototype as an intelligence upgrade to the control room they already use - same corridor names, same KM references, same terminology. Nothing is decommissioned.

What the layer adds

- A single physical view of work requested by three departments
- An explainable 0-100 priority score for every task
- Discovery of jobs that can share one possession
- A feasible schedule that respects trains, crew and isolation
- A record of why each recommendation was made

What we deliberately do not build

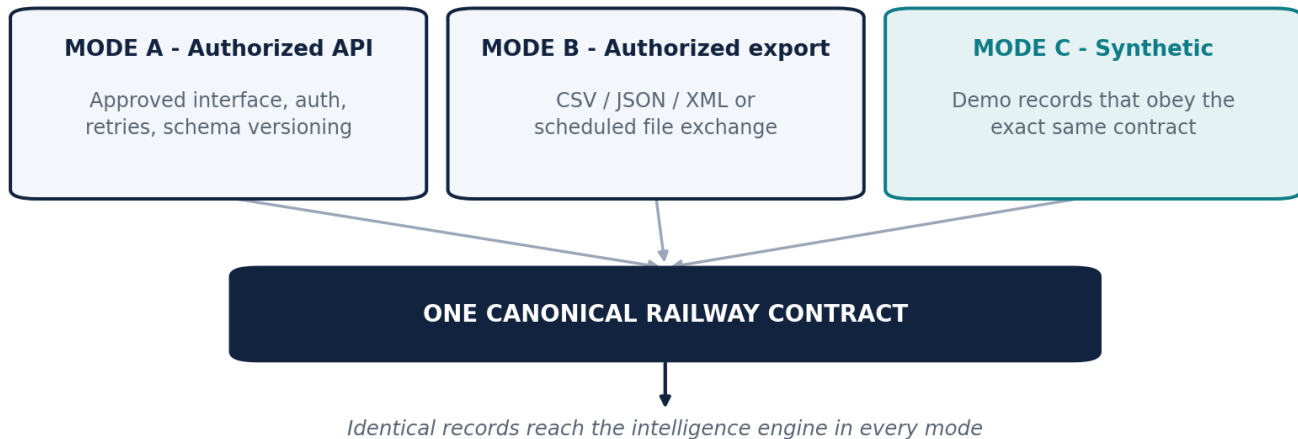
No replacement of BDMS | no full enterprise architecture | no unauthorised production API integration | no autonomous block approval | no LLM as the scheduling engine | no nationwide dataset. We build one narrow, convincing vertical slice.

What each stage has to produce

Stage	Purpose	Build target
Source adapters	Read the five departmental systems (API or file)	REST adapter stubs + CSV/JSON/XML importer
Integration gateway	Normalise source-specific schemas	One versioned canonical contract
Canonical topology	Map asset IDs and KM ranges onto one railway model	Corridor and asset mapping tables
Data health	Track freshness, validation and confidence	Timestamp and status per source
Risk engine	Rank work by safety, urgency, traffic and failure risk	Explainable 0-100 score
Auto-Shadow	Discover compatible cross-department work	Spatial + temporal + resource compatibility
CP-SAT	Select safe, feasible windows	Hard constraints plus soft objectives
Officer control centre	Explain, compare, simulate, approve or override	Control-room-style interface
Feedback loop	Compare plan against actual execution	Duration statistics for future planning

2. Data and integration: handling the biggest catch

We cannot assume production APIs will be available for TMS, SMMS, TDMS, COA or BDMS, and we must never demonstrate fake live Railway access. The gateway therefore supports three input modes that all produce identical canonical records, so synthetic data today can be swapped for an authorised source tomorrow without redesigning the intelligence engine.



Canonical model to lock early

Entity	Minimum fields	Why it matters
Asset	asset_id, department, type, corridor, km_from, km_to	Common physical identity
MaintenanceTask	task_id, asset_id, severity, overdue_days, duration, block_type, crew, equipment, depends_on	The scheduling job definition
Train	train_id, route, corridor, arrival, departure, priority	Traffic conflict model
BlockWindow	corridor, start, end, block_type, availability	Possession candidate
SourceMetadata	source, last_sync, schema_version, confidence	Freshness and auditability
TopologyMap	department asset reference -> canonical corridor + KM	Cross-system joins and bundle discovery

The synthetic railway division

- About 100 km of double-track corridor with roughly 10 stations
- Track, signal and OHE assets tied to KM positions
- Crews, equipment and their availability windows
- Passenger timetable plus freight forecasts
- Routine maintenance, overdue work and stochastic defects
- Deliberate cross-department tasks in the same window

Correlated, not random

Every record must have a reason to exist. If a signal defect appears at KM 140.2, there must be an S&T asset there, mapped to the same canonical corridor, with a duration, a crew and trains passing through it. A thousand random rows would collapse under the first hard question; a small coherent world proves coordination instead of merely displaying data.

One canonical record - from an API, a CSV export or the synthetic generator

```

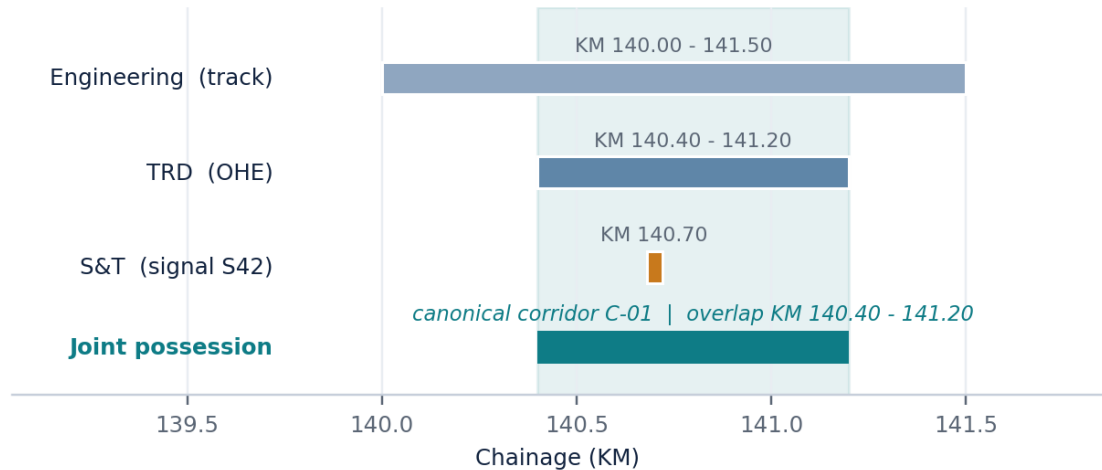
{
  "task_id": "SNT-2211", "source": "TDMS", "department": "S&T",
  "asset_id": "SIG-S42", "corridor": "C-01", "km_from": 140.7, "km_to": 140.7,
  "severity": "high", "overdue_days": 17, "est_duration_min": 60,
  "block_type": "traffic_block", "crew": ["SNT-C3"], "equipment": ["ME3"],
  "depends_on": [], "isolation_required": false,
  "source_meta": { "last_sync": "14:32", "schema_version": "1.0", "confidence": 0.94 }
}
  
```

3. Auto-Shadow Intelligence: the headline USP

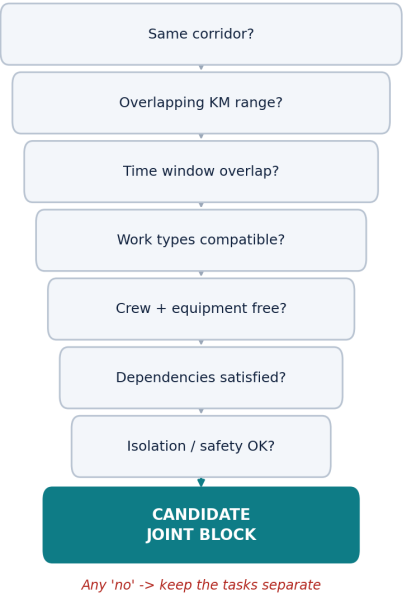
Three departments ask for possessions on the same corridor: Engineering 120 min, TRD 90 min, S&T 60 min. Handled independently that is 270 minutes of separate possession. Auto-Shadow detects that the work shares a corridor, a KM range and a feasible window, and proposes a **single 120-minute joint possession** - 150 minutes avoided in this illustrative scenario.

How the overlap is actually detected

The three departments describe the same stretch of track in three vocabularies: Track Section 17, OHE Section X and Signal Group 42. Once the topology layer resolves all three onto canonical corridor C-01, the shared KM band becomes arithmetic rather than guesswork.

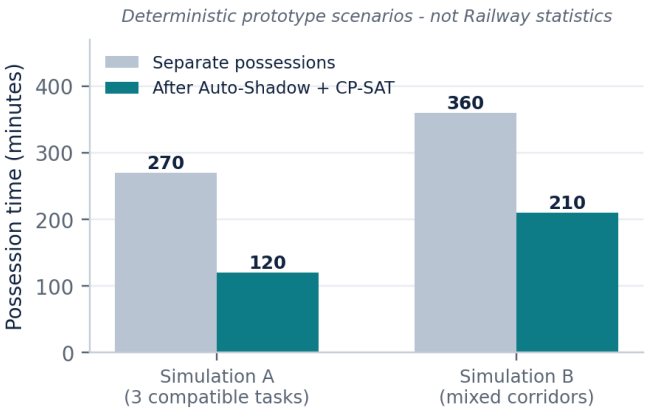


Compatibility gate



Overlap is not compatibility

The dangerous assumption would be *same corridor = combine everything*. Two tasks may sit on the same KM and still be impossible to run together: they may need the same crane, contradictory isolation states, or one may depend on the other finishing first. Bundling is therefore a deterministic filter, not an AI guess.

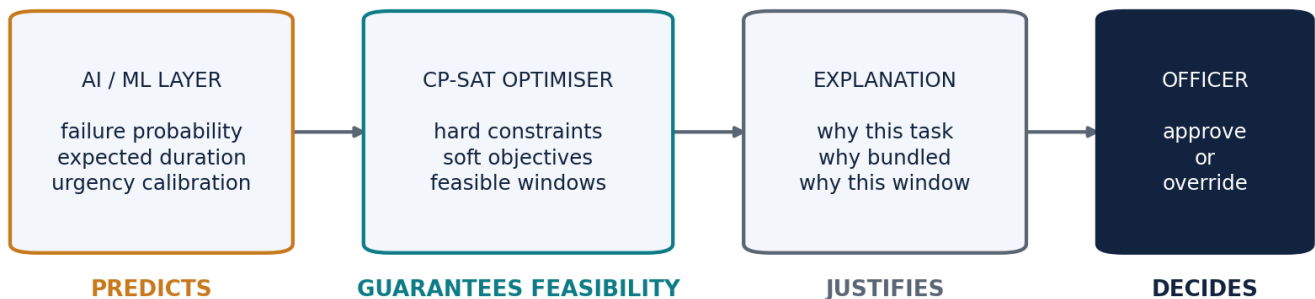


Simulation B mixes corridors on purpose: three compatible tasks on corridor A bundle, while a fourth Engineering task on corridor B needing the same crew and machine stays separate.

Simulation	Baseline	Optimised	Avoided	What it proves
A - three compatible tasks, one corridor	270 min	120 min	150 min (55.6%)	One window can carry compatible work from three departments
B - mixed corridors and resources	360 min	210 min	150 min (41.7%)	Same department or corridor alone is not enough to bundle

4. AI plus optimisation: two engines, not one big model

When judges ask "where is the AI?", the answer has to survive a follow-up about safety. Ours does: machine learning predicts the uncertain quantities, and a deterministic constraint solver decides the schedule. We never ask a probabilistic model to guarantee that a train and a possession do not collide.

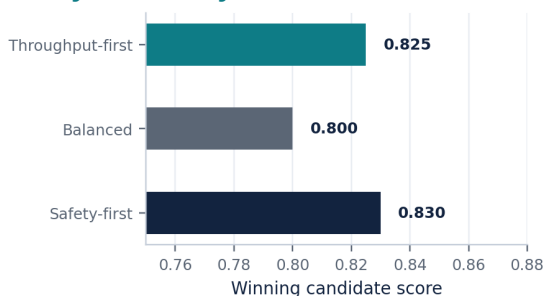


No autonomous block issuance at any point in the chain

Hard constraints versus soft objectives

Hard constraints - cannot be violated	Soft objectives - optimised
No incompatible train / block overlap	Minimise the number of possessions
Required isolation and safety rules	Maximise useful joint work
Crew, equipment and resource availability	Reduce train disruption
Task dependencies and sequencing	Reduce residual maintenance risk
Valid operating and traffic windows	Improve resource and block utilisation

Policy sensitivity



Weights are policy, not code

Running the same candidate set under safety-first, balanced and throughput-first weightings produces different winning scores (0.830 / 0.800 / 0.825). The optimiser never changes - only the declared soft-objective priorities do. That is why the weights belong in a configuration screen the division can own, not buried inside a Python file.

Keep the AI deliberately narrow

- Predict failure probability where history supports it
- Predict expected maintenance duration and resource time
- Calibrate urgency from overdue days and asset criticality
- Never place a general-purpose LLM between the officer and the scheduler

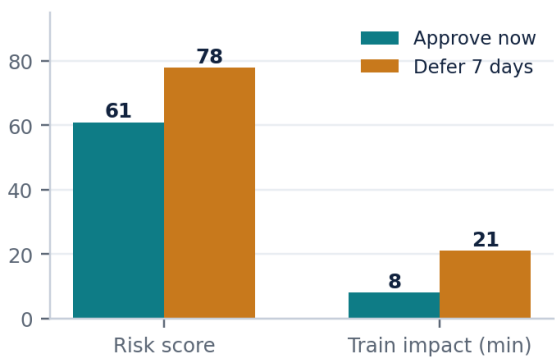
5. The officer experience: explain, simulate, decide

An officer should never receive a bare instruction like "take a block at 14:00". Every recommendation carries three explanations - why this task, why these departments were bundled, and why this window - plus a what-if comparison and an override. That is what turns a scheduler into a decision-support system.

WHY THIS TASK?		WHY BUNDLE?		WHY 02:00 - 04:00?	
Safety criticality	High	Engineering	KM 140.2 - 141.1	Train conflict	none
Overdue	17 days	TRD	KM 140.5 - 141.0	Crew available	yes
Asset criticality	High			Isolation possible	yes
Predicted failure probability	0.72	S&T	KM 140.7	Dependencies satisfied	yes
Traffic impact	Low	Same corridor	yes	Traffic impact	lowest of 6 windows
Priority score	87 / 100	Compatible resources	yes		
		Dependency conflict	none	Data snapshot	14:32 today

Every recommendation is auditable: the factors, the bundle rationale, the constraint checks and the freshness of the data it was based on.

What-if: the cost of waiting



A decision aid, not a prophecy

Approving now costs 8 minutes of train impact at a risk score of 61. Deferring seven days raises risk to 78 and train impact to 21 minutes, because the next feasible window is tighter. The simulator must always show its assumptions, its confidence and the next feasible window alongside the numbers.

Data health and graceful degradation

Source	Status	Effect
TMS	fresh	full confidence
SMMS	fresh	full confidence
TDMS	stale (last sync 14:32)	confidence 0.94, flagged in UI
COA	fresh	full confidence

The system continues on the last verified snapshot and says so. It never silently manufactures freshness.

6. Designing for the ugly cases

A demo that only survives the happy path loses the first question in the Q&A. Each failure mode below has a defined behaviour, and each one is worth rehearsing before the presentation.

Failure or edge case	System behaviour	Why it is defensible
Source API unavailable	Fall back to authorised export or the last verified snapshot; show stale status	The plan degrades visibly instead of failing silently
Different asset IDs across departments	Topology mapping resolves every reference to a canonical corridor and KM range	This is the integration problem, solved once
Two high-priority tasks clash	Optimiser searches alternatives; if none exist the officer gets an explicit conflict and escalation	The system never picks a winner in secret
No feasible schedule exists	Relax only declared soft constraints and report exactly what changed	Hard safety rules are never quietly bent
Train timetable changes	Freeze locked commitments and re-optimize only the affected horizon	Avoids rebuilding a plan the division already acted on
Emergency block arrives after publication	Insert as very high priority, re-optimize, and name the lower-priority task that moved	Change is traceable, not mysterious
Bad or incomplete source data	Validation rejects malformed records; confidence is reduced and shown	Garbage never reaches the optimiser unlabelled
Model prediction looks uncertain	Fall back to a conservative rule-based estimate and flag the uncertainty	The AI is allowed to say it does not know
Departments disagree on priority	Configurable policy weights plus officer escalation	Priority is an owned policy, not a hidden constant

The line we will not cross

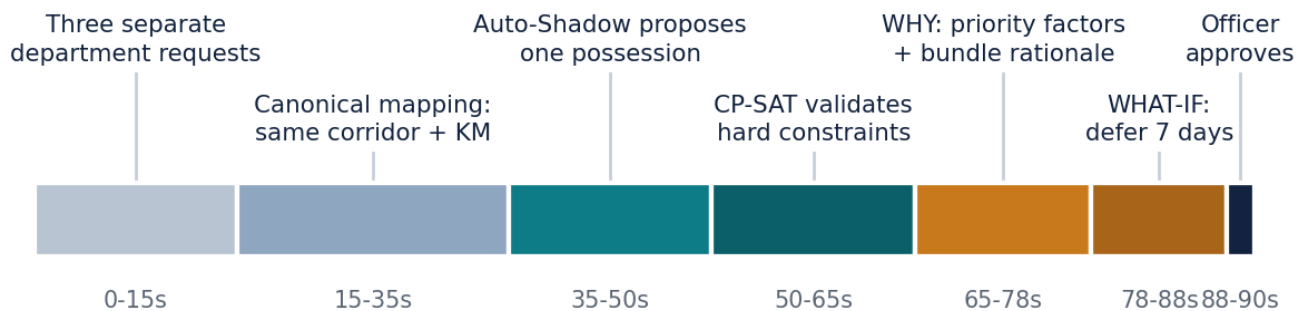
There is no autonomous block issuance anywhere in this design. The pipeline ends at a recommendation with an explanation; the officer approves or overrides it, and the approved block then flows back into the existing Railway workflow. Every approval and override is logged, which is what makes the plan-vs-actual learning loop possible in the first place.

7. Build blueprint and demonstration

P0 is the vertical slice that must work end to end. P1 items are genuine differentiators but they are worthless if the core pipeline is not running - adding them first would make the project weaker, not stronger.

Component	Definition of done
P0 Synthetic TMS / SMMS / TDMS / COA / BDMS	Cross-department tasks exist on one shared corridor
P0 Canonical model + integration gateway	API, file and synthetic inputs produce identical canonical records
P0 Topology and KM mapping	Every department reference resolves to a canonical corridor and KM range
P0 Risk and priority engine	Every task carries an explainable 0-100 score
P0 Auto-Shadow engine	Compatible tasks are discovered, bundled and justified
P0 CP-SAT scheduler	Produced schedule satisfies every hard constraint
P0 Officer dashboard	Why / What-if / Compare / Approve / Override all work
P1 Data health indicator	Fresh, stale and offline source status is visible
P1 Plan vs actual	Actual execution updates duration and resource statistics
P1 Extended rolling horizon	Monthly and strategic views beyond the weekly demo

90-SECOND DEMONSTRATION SCRIPT



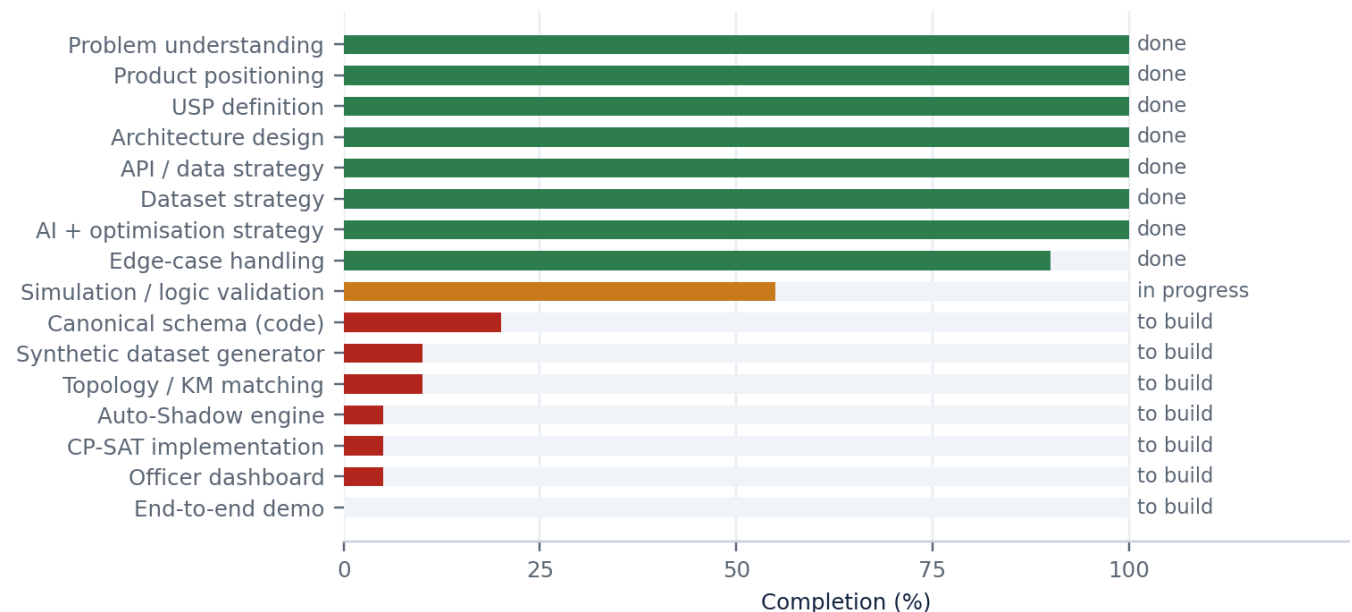
Closing line of the demo: **"We make existing Railway systems think together."**

Technical stack

Layer	Technology	Layer	Technology
Backend	Python + FastAPI	Optimisation	Google OR-Tools CP-SAT
Validation	Pydantic	Machine learning	Scikit-learn initially
Processing	Pandas	Frontend	React / Next.js
Database	PostgreSQL	Views	Corridor map, Gantt timeline, KPI cards
Integration	REST adapters + CSV/JSON/XML	Deployment	Docker Compose for the demo environment

8. Where the project stands today

Design and strategy are settled. What remains is engineering. The chart below is the honest status of each workstream - green is decided and documented, amber is prototype level, red is code that does not exist yet.



The phase change

We are past the "what should our project be?" stage. Further brainstorming now has a negative return: the strategy is already sharper than the implementation. The next milestone is a running pipeline, not another feature.

The seven remaining engineering battles, in build order



Highlighted steps carry the most engineering risk

#	Battle	The thing that must actually work
1	Canonical schema	Lock the JSON / table structure for Asset, Task, Train, Block, Crew, Equipment, Corridor and Source before anything else is written
2	Synthetic railway generator	Generate a 100 km world where every asset, task, defect, crew and train is logically related
3	Topology and KM matching	Resolve Engineering KM 140.0-141.5, TRD KM 140.4-141.2 and S&T KM 140.7 onto corridor C-01 and detect the overlap
4	Auto-Shadow algorithm	Define compatibility as a deterministic scoring and filtering rule set, not a vague AI judgement
5	CP-SAT formulation	Translate railway rules into variables, constraints and objectives that solve in seconds
6	Risk / ML layer	Only after the pipeline runs, decide precisely what is being predicted and from what data
7	Officer UI	Make the whole plan understandable to a railway officer in about ten seconds

9. The one-paragraph version

If you remember only one thing

This is not an AI block scheduler. It is a **cross-department railway intelligence layer** that converts fragmented maintenance requests into a common physical view, discovers work that can be performed together, predicts risk, uses deterministic optimisation to build a safe feasible plan, and leaves the final decision with the Railway officer who already owns it.

One corridor. One coordinated possession. Maximum useful work.

We make existing Railway systems think together.

Answers to the four questions we will definitely be asked

Question	Answer in one breath
Where exactly is the AI?	In prediction - failure probability, maintenance duration and urgency calibration. Scheduling is done by a deterministic CP-SAT solver, because a probabilistic model cannot guarantee that a train and a possession never overlap.
How will you get Railway data?	Through an adapter gateway with three modes. Authorised API if it is granted, authorised file export as the realistic fallback, and synthetic data for this prototype. All three produce identical canonical records, so nothing downstream changes when the source changes.
What if the AI is wrong?	It cannot issue a block. Every recommendation carries its factors, its constraints and its data snapshot, and the officer approves or overrides. Uncertain predictions fall back to conservative rule-based estimates and are flagged.
Why would Railways adopt this?	Because nothing is replaced. TMS, SMMS, TDMS, COA and BDMS stay exactly where they are; the layer above them simply notices that three departments want the same stretch of track and proposes one possession instead of three.

Final verdict

Do not build a generic AI dashboard. Build a narrow vertical slice that proves one difficult operational truth: different departments can keep using their existing systems while a new intelligence layer discovers shared work, prioritises it, safely schedules it, and leaves an officer in control.

Scope note. All numerical examples in this report - possession minutes, risk scores, confidence values and policy scores - are deterministic prototype scenarios created to test architecture behaviour and compare design choices. They are not measured Indian Railways operational statistics and must not be presented as such. Production API contracts, authentication, live schemas, deployment and security requirements still require confirmation with authorised Railway stakeholders.